

Chaînes de Markov cachées

Maxime Delpéché et Marius Requillart

2026

1 Introduction

Définition d'une chaîne de Markov cachée

États cachés. Les états cachés sont modélisés par un processus de Markov $(X_t)_{t \in \mathbb{N}}$ à valeurs dans un ensemble fini

$$E = \{1, \dots, N\}.$$

On note $A = (a_{ij})$ la matrice de transition, avec

$$a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i).$$

Observations. Les observations sont modélisées par un processus $(Y_t)_{t \in \mathbb{N}}$ à valeurs dans un ensemble O . Conditionnellement aux états cachés :

$$\mathbb{P}(Y_t = y \mid X_t = i) = b_i(y),$$

où $B = (b_i(y))$ est la matrice, ou famille, d'émission.

Modèle HMM. Un modèle de chaîne de Markov cachée est défini par le triplet

$$(A, B, \pi),$$

où A est la matrice de transition, B la loi d'émission et π la distribution initiale.

Exemple global d'une chaîne de Markov cachée

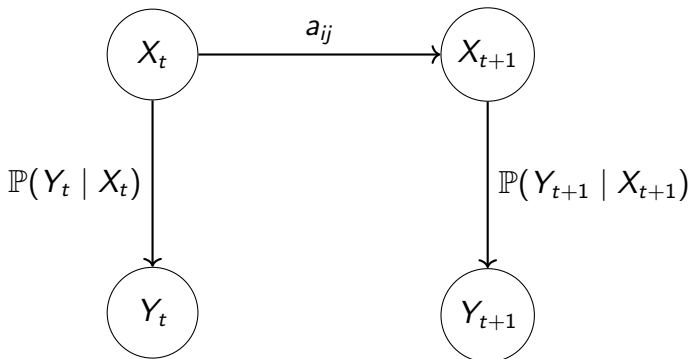
$$E = \{C, V\}, \quad \pi_0 = (0.5, 0.5)$$

$$A = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$$

$$a_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i)$$

$$B = \begin{cases} \mathcal{L}(Y_t \mid X_t = C) \sim \mathcal{N}(\mu_C, \sigma_C^2), \\ \mathcal{L}(Y_t \mid X_t = V) \sim \mathcal{N}(\mu_V, \sigma_V^2). \end{cases}$$

Structure du modèle HMM



Les états cachés évoluent selon A , et chaque observation est générée conditionnellement à l'état courant.

Les observations ne sont pas markoviennes

Propriété. Dans un HMM non dégénéré, le processus d'observations $(Y_t)_t$ n'est pas un processus de Markov, même si les états cachés $(X_t)_t$ le sont.

On a :

$$\mathbb{P}(Y_{n+1} \mid Y_1, \dots, Y_n) = \sum_{i=1}^K \mathbb{P}(Y_{n+1} \mid X_{n+1} = i) \mathbb{P}(X_{n+1} = i \mid Y_1, \dots, Y_n)$$

Or,

$$\mathbb{P}(X_{n+1} = i \mid Y_1, \dots, Y_n) = \sum_{j=1}^K \mathbb{P}(X_{n+1} = i, X_n = j \mid Y_1, \dots, Y_n)$$

Et

$$\mathbb{P}(X_{n+1} = i \mid Y_1, \dots, Y_n) = \sum_{j=1}^K a_{ji} \mathbb{P}(X_n = j \mid Y_1, \dots, Y_n)$$

Dépendance au passé observé

$$\mathbb{P}(X_n = j \mid Y_1, \dots, Y_n) = \frac{\mathbb{P}(Y_n \mid X_n = j, Y_1, \dots, Y_{n-1})\mathbb{P}(X_n = j, Y_1, \dots, Y_{n-1})}{\mathbb{P}(Y_1, \dots, Y_n)}$$

$$\mathbb{P}(X_n = j \mid Y_1, \dots, Y_n) = \frac{\mathbb{P}(Y_n \mid X_n = j)\mathbb{P}(X_n = j \mid Y_1, \dots, Y_{n-1})}{\mathbb{P}(Y_n \mid Y_1, \dots, Y_{n-1})}.$$

$$\begin{aligned}\mathbb{P}(Y_{n+1} \mid Y_1, \dots, Y_n) &= \sum_{i=1}^K \sum_{j=1}^K a_{ji} \frac{\mathbb{P}(Y_{n+1} \mid X_{n+1} = i)\mathbb{P}(Y_n \mid X_n = j)}{\mathbb{P}(Y_n \mid Y_1, \dots, Y_{n-1})} \\ &\quad \times \mathbb{P}(X_n = j \mid Y_1, \dots, Y_{n-1}).\end{aligned}$$

Ainsi, on remarque bien que

$$\mathbb{P}(Y_{n+1} \mid Y_1, \dots, Y_n) \neq \mathbb{P}(Y_{n+1} \mid Y_n).$$

Algorithme Forward

La vraisemblance de la séquence observée est :

$$\mathbb{P}(Y) = \sum_X \mathbb{P}(Y, X) = \sum_X \mathbb{P}(Y | X) \mathbb{P}(X).$$

Principe

On définit

$$\alpha_t(i) = \mathbb{P}(y_1, \dots, y_t, X_t = i).$$

Initialisation

$$\alpha_1(i) = \pi_i b_i(y_1), \quad i = 1, \dots, N.$$

Récursion

$$\alpha_t(j) = b_j(y_t) \sum_{i=1}^N \alpha_{t-1}(i) a_{ij}, \quad t = 2, \dots, T.$$

Terminaison

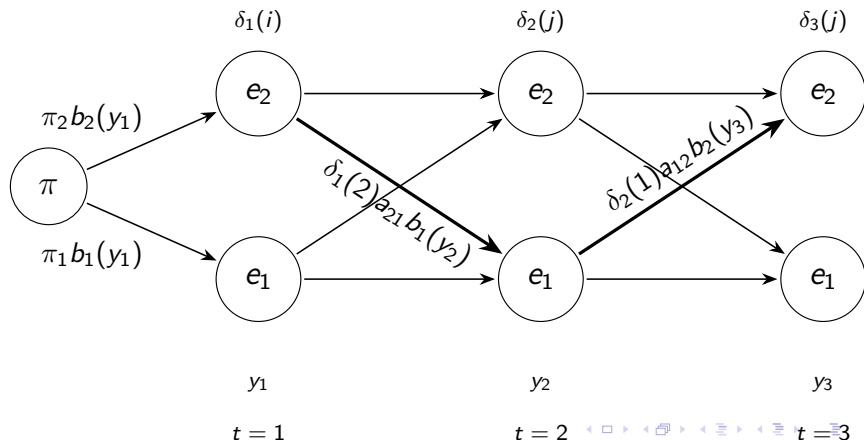
$$\mathbb{P}(Y) = \sum_{i=1}^N \alpha_T(i).$$

Algorithme de Viterbi

On estime le chemin le plus probable avec :

$$\delta_t(i) = \max_{x_1, \dots, x_{t-1}} \mathbb{P}(X_1, \dots, X_t = i, y_1, \dots, y_t \mid \lambda)$$

$$\delta_{t+1}(j) = \left[\max_{1 \leq i \leq N} \delta_t(i) a_{ij} \right] b_j(y_{t+1})$$



Algorithme de Baum–Welch : étape E

Baum–Welch est un algorithme EM : à partir d'un modèle courant

$$\lambda^{(k)} = (\pi^{(k)}, A^{(k)}, B^{(k)}),$$

on estime les états cachés, puis on met à jour les paramètres.

Probabilité d'être dans l'état i au temps t :

$$\gamma_t(i) = \mathbb{P}(X_t = i \mid Y, \lambda^{(k)}) = \frac{\alpha_t(i)\beta_t(i)}{\sum_{\ell=1}^N \alpha_t(\ell)\beta_t(\ell)}.$$

Probabilité de transition $i \rightarrow j$ entre t et $t+1$:

$$\xi_t(i, j) = \mathbb{P}(X_t = i, X_{t+1} = j \mid Y, \lambda^{(k)}).$$

$$\xi_t(i, j) = \frac{\alpha_t(i)a_{ij}^{(k)}b_j^{(k)}(y_{t+1})\beta_{t+1}(j)}{\sum_{r=1}^N \sum_{s=1}^N \alpha_t(r)a_{rs}^{(k)}b_s^{(k)}(y_{t+1})\beta_{t+1}(s)}.$$

$$\gamma_t(i) \quad \text{et} \quad \xi_t(i, j)$$

donnent les nombres moyens de visites des états et de transitions.

Algorithme de Baum–Welch : étape M

Une fois $\gamma_t(i)$ et $\xi_t(i, j)$ calculés, on met à jour les paramètres.

Distribution initiale :

$$\pi_i^{(k+1)} = \gamma_1(i).$$

Matrice de transition :

$$a_{ij}^{(k+1)} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}.$$

Émissions discrètes :

$$b_j^{(k+1)}(y) = \frac{\sum_{t=1}^T \gamma_t(j) \mathbf{1}_{\{Y_t=y\}}}{\sum_{t=1}^T \gamma_t(j)}.$$

Émissions gaussiennes :

$$\mu_j^{(k+1)} = \frac{\sum_{t=1}^T \gamma_t(j) Y_t}{\sum_{t=1}^T \gamma_t(j)}, \quad (\sigma_j^2)^{(k+1)} = \frac{\sum_{t=1}^T \gamma_t(j) (Y_t - \mu_j^{(k+1)})^2}{\sum_{t=1}^T \gamma_t(j)}.$$

On répète ces étapes jusqu'à stabilisation de

$$\ell^{(k)} = \log \mathbb{P}(Y \mid \lambda^{(k)}).$$

Évaluation de l'algorithme Forward

L'algorithme Forward permet d'évaluer la qualité du modèle à partir de la vraisemblance :

$$\ell(\lambda) = \log \mathbb{P}(Y_{1:T} \mid \lambda).$$

Critères utilisés :

- **Log-vraisemblance moyenne**

$$\bar{\ell}(\lambda) = \frac{1}{T} \ell(\lambda)$$

- **Critères pénalisés**

$$AIC = -2\ell(\lambda) + 2p, \quad BIC = -2\ell(\lambda) + p \log(T)$$

- **Validation hors échantillon**

$$NLPD_{\text{test}} = -\frac{1}{T_{\text{test}}} \log \mathbb{P}(Y_{\text{test}} \mid \hat{\lambda}_{\text{train}})$$

Une bonne performance Forward correspond à une log-vraisemblance élevée, des critères AIC/BIC faibles, et une NLPD stable sur les données de test.

Évaluation de l'algorithme de Viterbi

Viterbi estime la trajectoire cachée la plus probable :

$$\hat{X}_{1:T}^V \in \arg \max_{x_{1:T}} \mathbb{P}(X_{1:T} = x_{1:T} \mid Y_{1:T}, \hat{\lambda}).$$

Critères utilisés :

- **Interprétation des régimes** : vérifier si les états détectés correspondent à des comportements observables distincts.
- **Persistance des régimes** :

$$\text{taux de changement} = \frac{1}{T-1} \sum_{t=1}^{T-1} \mathbf{1}_{\{\hat{X}_t^V \neq \hat{X}_{t+1}^V\}}.$$

- **Comparaison avec le posterior decoding**

$$D_{VP} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}_{\{\hat{X}_t^V \neq \hat{X}_t^P\}}.$$

Un bon décodage est cohérent temporellement, interprétable, et proche du posterior decoding lorsque l'incertitude sur les états est faible.

Évaluation de l'algorithme de Baum–Welch

Baum–Welch estime les paramètres du modèle :

$$\hat{\lambda} = (\hat{\pi}, \hat{A}, \hat{B}) \in \arg \max_{\lambda} \mathbb{P}(Y_{1:T} \mid \lambda).$$

Critères utilisés :

- **Convergence de la log-vraisemblance**

$$\Delta_k = \ell^{(k)} - \ell^{(k-1)}, \quad r_k = \frac{\Delta_k}{1 + |\ell^{(k-1)}|}.$$

- **Stabilité selon l'initialisation**

$$D_A = \frac{1}{N} \sum_i \frac{1}{2} \sum_j |\hat{a}_{ij}^{(a)} - \hat{a}_{ij}^{(b)}|.$$

- **Interprétation des régimes** : les états estimés doivent correspondre à des comportements distincts.
- **Validation hors échantillon**

$$NLDP_{\text{test}} = -\frac{1}{T_{\text{test}}} \log \mathbb{P}(Y_{\text{test}} \mid \hat{\lambda}_{\text{train}}).$$

Simulation d'une chaîne de Markov cachée

On simule un HMM à trois états cachés :

$$S = \{\text{soleil, nuageux, pluie}\}$$

et deux observations possibles :

$$O = \{\text{parapluie, pas parapluie}\}.$$

$$\pi = (0.5 \quad 0.3 \quad 0.2)$$

$$A = \begin{pmatrix} 0.7 & 0.295 & 0.005 \\ 0.3 & 0.4 & 0.3 \\ 0.005 & 0.395 & 0.6 \end{pmatrix} \quad B = \begin{pmatrix} 0.1 & 0.9 \\ 0.4 & 0.6 \\ 0.9 & 0.1 \end{pmatrix}$$

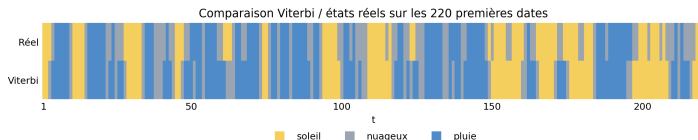
La trajectoire simulée contient $T = 1001$ observations, avec les vrais états cachés connus.

Résultats bruts des algorithmes sur la simulation

Forward

$$\log \mathbb{P}(Y_{1:T} \mid \lambda) = -639.814329.$$

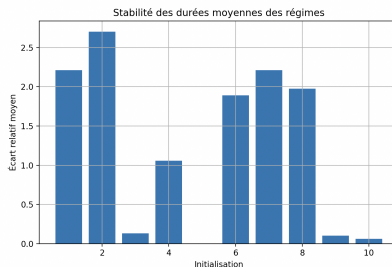
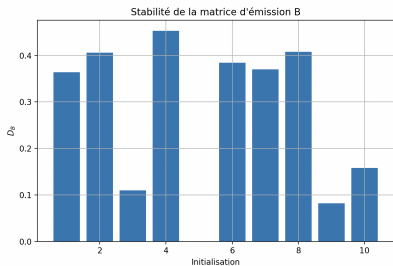
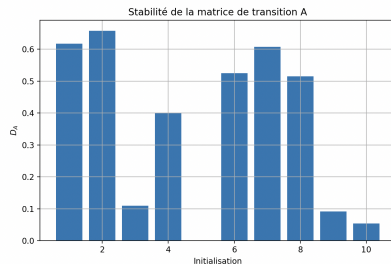
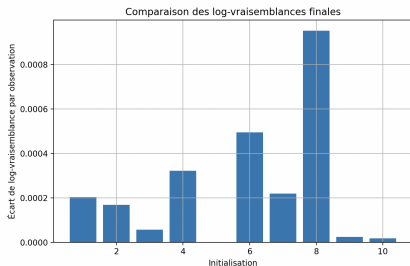
Viterbi



Baum–Welch

$$\hat{A} = \begin{pmatrix} 0.7026 & 0.2956 & 0.0018 \\ 0.2720 & 0.5963 & 0.1316 \\ 0.1804 & 0.2172 & 0.6024 \end{pmatrix} \quad \hat{B} = \begin{pmatrix} 0.1070 & 0.8930 \\ 0.5702 & 0.4298 \\ 0.9895 & 0.0105 \end{pmatrix}$$

Analyse Baum–Welch sur la simulation



Application au VIX

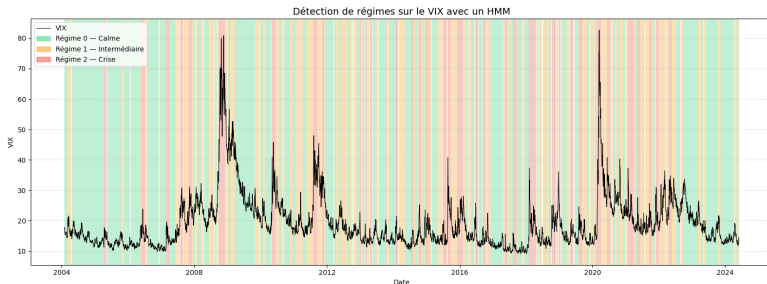
On applique le HMM au VIX en utilisant trois variables observées :

$$R_t = \log(V_t) - \log(V_{t-1}), \quad \sigma_t^{(5)} = \text{std}(R_{t-4}, \dots, R_t), \quad \sigma_t^{(20)} = \text{std}(R_{t-19}, \dots, R_t).$$

Le rendement logarithmique R_t décrit les variations journalières du VIX, la volatilité courte $\sigma_t^{(5)}$ capte les mouvements récents, et la volatilité longue $\sigma_t^{(20)}$ décrit une dynamique plus persistante.

Statistiques moyennes obtenues pour les trois régimes

Régime	VIX moyen	R_t moyen	$\sigma_t^{(20)}$ moyenne	Nombre d'observations
Calme	16.3809	0.0007	0.0466	2364
Intermédiaire	20.0225	-0.0021	0.0719	1751
Crise	23.6703	0.0018	0.1134	1003

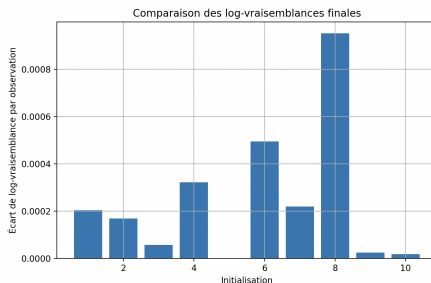


Analyse Baum–Welch sur le VIX

Convergence

Critère	Valeur
Nombre d'observations T	5118
Meilleure initialisation	10
Nombre d'itérations	36
Log-vraisemblance finale	-14300.087178
Log-vraisemblance moyenne	-2.794077
Gain final / observation	9.728857×10^{-11}
Gain relatif final	3.481714×10^{-11}
Baisses détectées	0

Comparaison des initialisations



Validation hors échantillon

Segment	T	Log-vraisemblance	NLPD
Apprentissage	4094	-11536.441391	2.817890
Test conditionnel	1024	-2774.008272	2.708992
Test indépendant	1024	-2782.320895	2.717110